

TRABAJO PRÁCTICO INTEGRADOR

Sistemas de Semáforos Inteligentes

Análisis comparativo: Common Lisp y Erlang

Grupo 45

*Integrantes: Rudas, Emilia. Acosta,
Gonzalo David. Panzich, Luca. Navas,
Tiziano. Palmeyro, Patricio.*

Fecha de entrega: 16/06/2026

ÍNDICE

INTRODUCCIÓN	4
FASE 1: SISTEMA DE SEMÁFOROS INTELIGENTES	5
Descripción General	5
Requerimiento 1: Estados de Transición	5
Requerimiento 2: Temporizador Automático	5
Requerimiento 3: Sistema de Auditoría	6
Requerimiento 4: Análisis de Ciclos Semafóricos	6
Requerimiento 5: Planificación Temporal.....	7
Requerimiento 6: Distribución Temporal.....	7
CLASIFICACIÓN DE LAS FUNCIONES IMPLEMENTADAS.....	7
BITÁCORA DE DEPURACIÓN	9
FASE 2: INVESTIGACIÓN E INTEGRACIÓN DE LIBRERÍAS EXTERNAS	13
Introducción.....	13
Análisis de local-time	13
Análisis de cl-json	14
Comparativa de Factores Operativos	15
Decisión Técnica	16
FASE 3: ESTUDIO COMPARATIVO – ERLANG.....	17
Introducción al Lenguaje	17
Áreas de Aplicación	17
Empresas que Utilizan Erlang	18
Implementación de la función transicion en Erlang.....	18
Implementación de la función timer en Erlang.....	19
Delimitación de Funciones y Expresiones en Erlang	19
Variables de Asignación Única y Simulación de un Ciclo Continuo	20
Comparación con Common Lisp.....	21
Conclusión	22
CONCLUSIÓN GENERAL.....	23

BIBLIOGRAFÍA.....	24
--------------------------	-----------

INTRODUCCIÓN

El presente Trabajo Práctico Integrador tiene como objetivo aplicar los conceptos fundamentales de programación funcional mediante el desarrollo de un Sistema de Semáforos Inteligentes en Common Lisp.

La propuesta permite modelar una problemática real relacionada con la gestión del tránsito urbano, implementando funciones que controlan estados semafóricos, temporización automática, auditoría de eventos, análisis de ciclos y distribución temporal de los distintos estados de una intersección.

Durante el desarrollo se aplicaron principios característicos del paradigma funcional, tales como la inmutabilidad de los datos, el uso de funciones puras, la ausencia de estados globales mutables y el diseño basado en composición funcional.

Además del desarrollo del sistema, se investigaron herramientas pertenecientes al ecosistema de Common Lisp mediante la utilización de Quicklisp y se realizó un estudio comparativo con otro lenguaje funcional asignado por la cátedra.

FASE 1: SISTEMA DE SEMÁFOROS INTELIGENTES

Descripción General

El sistema desarrollado simula el comportamiento de un semáforo urbano mediante la implementación de diversos requerimientos funcionales.

La solución permite representar los cambios de estado de una luz de tránsito, automatizar su funcionamiento mediante temporizadores, registrar eventos para auditoría y realizar análisis estadísticos sobre los ciclos semafóricos.

Requerimiento 1: Estados de Transición

Se implementó una función denominada *transicion*, encargada de modelar el cambio de estados del semáforo.

La función recibe el estado actual del semáforo y el estado al que se desea realizar la transición. A partir de estos valores devuelve una estructura indicando la acción correspondiente al cambio solicitado. La versión final incorpora estados de intermitencia entre cada color principal, permitiendo representar con mayor precisión el comportamiento real del semáforo. En caso de recibir una transición inválida, retorna una acción por defecto.

Requerimiento 2: Temporizador Automático

Se implementó una función denominada *timer*, encargada de determinar el estado del semáforo en un instante específico expresado mediante tiempo Unix (Epoch).

A diferencia de la primera versión del sistema, la Iteración 2 incorpora estados de intermitencia de tres segundos entre cada transición, simulando con mayor precisión el comportamiento de un semáforo real.

Las duraciones definidas para cada estado son:

- Rojo: 90 segundos
- Intermitente rojo: 3 segundos
- Verde: 120 segundos
- Intermitente verde: 3 segundos

- Amarillo: 6 segundos
- Intermitente amarillo: 3 segundos

La función calcula la posición del instante dentro del ciclo completo mediante la operación módulo y determina automáticamente qué estado corresponde a ese momento.

La duración total del ciclo es de 225 segundos.

Requerimiento 3: Sistema de Auditoría

Se desarrolló un mecanismo de registro que permite almacenar cada cambio de estado producido durante la ejecución.

El objetivo principal es posibilitar la reconstrucción histórica del comportamiento del semáforo y facilitar futuras tareas de auditoría o análisis forense.

Formato utilizado:

Tiempo <epoch>: la luz ha cambiado de <color-anterior> a <color-nuevo>

Requerimiento 4: Análisis de Ciclos Semafóricos

Se desarrollaron funciones destinadas al análisis de la duración de los ciclos semafóricos y a la generación de recomendaciones sobre su configuración.

Un ciclo completo se considera finalizado cuando el sistema regresa nuevamente al estado rojo luego de atravesar todas las etapas del semáforo:

Rojo → Intermitente Rojo → Verde → Intermitente Verde → Amarillo → Intermitente Amarillo → Rojo

La función duracion-ciclo permite calcular la duración total de un ciclo a partir de los tiempos configurados para cada color.

Por su parte, la función recomendacion-ciclo evalúa si la duración obtenida se encuentra dentro del rango recomendado por la consigna, considerando que ciclos menores a 35 segundos o mayores a 150 segundos pueden resultar poco adecuados para los usuarios de la vía pública.

Requerimiento 5: Planificación Temporal

Se desarrolló una función que permite determinar cuántos ciclos completos se ejecutan dentro de un intervalo de tiempo determinado.

Esta funcionalidad resulta útil para tareas de planificación, simulación y análisis de capacidad.

Requerimiento 6: Distribución Temporal

Se implementó una función encargada de calcular el porcentaje de tiempo que cada color permanece activo durante un período determinado.

Para ello se desarrollaron funciones auxiliares que contabilizan de manera recursiva la cantidad de segundos correspondientes a cada estado dentro de los ciclos completos ejecutados durante el intervalo analizado.

Posteriormente se aplica una regla de tres simple para obtener el porcentaje de tiempo asociado a cada color.

Esta información resulta útil para analizar el comportamiento general del sistema y evaluar posibles ajustes orientados a mejorar la circulación vehicular.

CLASIFICACIÓN DE LAS FUNCIONES IMPLEMENTADAS

Función	Naturaleza	Estrategia	Impacto
Transicion	Pura	Condicional	No destructiva
obtener-timeexac	Impura	Orden Superior	No destructiva
Timer	Pura	Condicional	No destructiva
registro-semaforo	Impura	Secuencial	No destructiva

Función	Naturaleza	Estrategia	Impacto
duracion-ciclo	Pura	Orden Superior	No destructiva
recomendacion- ciclo	Pura	Condicional	No destructive
ciclos-por- tiempo	Pura	Secuencial	No destructive
Porcentaje	Pura	Orden Superior	No destructive
contador-rojo	Pura	Recursiva	No destructive
contador- amarillo	Pura	Recursiva	No destruktiva
contador-verde	Pura	Recursiva	No destruktiva
Informe	Impura	Orden Superior	Destructiva
formatear-fecha	Pura	Secuencial	No destruktiva

La clasificación anterior permite identificar el comportamiento funcional de cada componente desarrollado y verificar el cumplimiento de los principios exigidos por la consigna.

BITÁCORA DE DEPURACIÓN

Durante el desarrollo del proyecto surgieron diversos errores que requirieron análisis y corrección. La siguiente bitácora documenta los problemas encontrados y las soluciones implementadas.

Error N.º 1: Uso incorrecto de LET

Descripción

Durante la implementación del temporizador se utilizó la construcción LET para definir variables dependientes entre sí.

Causa

La estructura LET evalúa todas las asignaciones simultáneamente, por lo que las variables definidas previamente todavía no se encontraban disponibles para las siguientes expresiones.

Solución

Reemplazar la función por let*, la cual realiza una lectura secuencial (de arriba hacia abajo). Esto permite que cada nueva variable pueda utilizar los valores de las variables definidas en las líneas anteriores.

Evidencia

```

Break 7 [9]> (defun obtener-timeexac ()
(- (get-universal-time) 2208988800))

(defun timer(time-exac)
  (let (
    (duracion-rojo 90)
    (duracion-intermitente-rojo 3)
    (duracion-verde 120)
    (duracion-intermitente-verde 3)
    (duracion-amarillo 6)
    (duracion-intermitente-amarillo 3)
    (ciclo-total (+ duracion-rojo duracion-intermitente-rojo duracion-verde duracion-intermitente-verde
duracion-amarillo duracion-intermitente-amarillo))
    (segundo-actual (mod time-exac ciclo-total))
  )
  (cond
    ((< segundo-actual 90) 'rojo)
    ((< segundo-actual 93) 'intermitente-rojo)
    ((< segundo-actual 213) 'verde)
    ((< segundo-actual 216) 'intermitente-verde)
    ((< segundo-actual 222) 'amarillo)
    (t 'intermitente-amarillo)
  )
)
)

```

```

OBTENER-TIMEEXAC
Break 7 [9]>
TIMER
Break 7 [9]> (timer 1222239933)

*** - LET: variable DURACION-ROJO has no value
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of DURACION-ROJO.
STORE-VALUE    :R2      Input a new value for DURACION-ROJO.
ABORT          :R3      Abort debug loop
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort debug loop
ABORT          :R7      Abort debug loop
ABORT          :R8      Abort debug loop
ABORT          :R9      Abort debug loop

```

Error N.º 2: Porcentajes expresados como fracciones

Descripción

Al calcular los porcentajes de tiempo de cada color, Common Lisp devolvía fracciones racionales en lugar de números decimales.

Causa

Las divisiones entre enteros generan resultados racionales exactos.

Solución

Envolver la operación matemática con la función primitiva float para forzar la conversión de tipo.

Evidencia

```
PORCENTAJE-ROJO
Break 7 [9]> (defun porcentaje-rojo (hora)
  (/ (* (contador-rojo hora) 100) 3600)
)

PORCENTAJE-ROJO
Break 7 [9]> (porcentaje-rojo 3600)

85/2
```

Error N.º 3: Resultado sobrante en ciclos completos

Descripción

La función encargada de calcular ciclos completos devolvía información adicional relacionada con el resto de la división.

Causa

El cálculo retornaba valores que no formaban parte del resultado solicitado.

Solución

Agregar la función `Values` esta se encarga de eliminar el sobrante que queda y solo dejar el entero a modo tal de que solo se reconocen los ciclos completos.

Evidencia

```
Break 8 [10]> (defun ciclos-por-tiempo (minutos)
  (truncate (* minutos 60) 225)
)

CICLOS-POR-TIEMPO
Break 8 [10]> (ciclos-por-tiempo 60)

16 ;
0
```

Error N.º 4: Conteo incorrecto del tiempo verde

Descripción

La función que contabilizaba el tiempo verde sumaba segundos inexistentes cuando el intervalo analizado finalizaba antes de alcanzar dicho estado.

Causa

La recursividad continuaba evaluándose aun cuando no existía tiempo suficiente para llegar al color verde.

Solución

Se agregó una barrera en el cond: `((<= hora 93) 0)`. Donde esto frena la evaluación antes de llegar a la recursividad y de ese modo no suma.

Evidencia

```
Break 7 [9]> (defun contador-verde (hora)
  (cond
    ((<= hora 0) 0)
    ((and (> hora 93) (< hora 216)) (- hora 93))
    ((> hora 0) (+ 123 (contador-verde (- hora 225)))))
  )
)

CONTADOR-VERDE
Break 7 [9]> (contador-verde 3600)

1968
Break 7 [9]> (contador-verde 50)

123
```

FASE 2: INVESTIGACIÓN E INTEGRACIÓN DE LIBRERÍAS EXTERNAS

Introducción

Como parte de la segunda fase del proyecto se investigó el uso de Quicklisp como gestor de paquetes para Common Lisp y se analizaron las librerías propuestas por la cátedra: local-time y cl-json.

El objetivo fue determinar cuál de ellas aporta una mejora más significativa al Sistema de Semáforos Inteligentes considerando criterios de mantenibilidad, escalabilidad y utilidad práctica.

Análisis de local-time

La librería local-time mejora la legibilidad de los registros generados por el sistema.

Su principal aporte consiste en transformar el tiempo Unix (Epoch) en una representación legible para los usuarios.

Por ejemplo:

[2026-06-12 14:30:15]

La librería local-time cumple con el objetivo de mejorar la legibilidad para el análisis forense, permitiendo una reconstrucción histórica de estados más eficiente.

Al transformar el tiempo Unix en un formato legible, se elimina la necesidad de que los peritos conviertan manualmente números de diez dígitos para identificar el color de la luz en el momento exacto de un suceso.

Además, una librería especializada permite gestionar correctamente aspectos como zonas horarias y años bisiestos, facilitando la interpretación de los eventos registrados.

Ventajas

- Mayor legibilidad.
- Facilita auditorías.
- Mejora el análisis forense.

- Reduce errores de interpretación.

Análisis de cl-json

La librería cl-json permite cargar configuraciones desde archivos externos en formato JSON.

En lugar de definir los temporizadores directamente dentro del código:

Rojo = 90

Amarillo = 6

Verde = 120

es posible utilizar un archivo externo:

```
{
  "rojo": 90,
  "amarillo": 6,
  "verde": 120
}
```

Permite que los tiempos de temporización no estén fijos en el código, sino que se carguen desde un archivo de configuración.

Esta capacidad resulta fundamental para la optimización del flujo vehicular, ya que no todos los semáforos de una ciudad poseen los mismos temporizadores y las variaciones son constantes debido a obras, demanda estacional o planificación de mantenimiento.

Desde la perspectiva de la ingeniería de software, esto representa un desacoplamiento entre datos y lógica, permitiendo reutilizar el mismo núcleo lógico en múltiples intersecciones modificando únicamente el archivo de configuración.

Ventajas

- Mayor mantenibilidad.
- Mayor escalabilidad.
- Configuración dinámica.
- Reutilización del software.
- Menor costo de mantenimiento.

Comparativa de Factores Operativos

Al contrastar ambas alternativas se observa que cada una resuelve necesidades diferentes dentro del sistema.

La librería `cl-json` aporta ventajas relacionadas con la mantenibilidad y la configuración dinámica de los temporizadores, permitiendo modificar parámetros operativos sin intervenir directamente sobre el código fuente. Esta característica resulta útil en entornos donde las configuraciones cambian con frecuencia entre distintas intersecciones.

Por otro lado, la librería `local-time` mejora significativamente la calidad de los registros generados por el sistema, permitiendo transformar los tiempos Unix en fechas y horarios comprensibles para los usuarios. Esta funcionalidad adquiere especial importancia en sistemas donde la auditoría, la trazabilidad y la reconstrucción histórica de eventos constituyen requisitos fundamentales.

En el contexto del presente proyecto, uno de los objetivos principales consiste en poder determinar con precisión qué estado tenía el semáforo en un instante determinado y disponer de registros fácilmente interpretables para posteriores análisis. La posibilidad de visualizar directamente la fecha y hora de cada transición simplifica considerablemente las tareas de monitoreo, auditoría y análisis forense.

Si bien ambas librerías aportan beneficios relevantes, se consideró que la mejora en la legibilidad y confiabilidad de los registros ofrecida por `local-time` representa un aporte más directamente relacionado con los requerimientos funcionales del sistema desarrollados en este trabajo.

Por esta razón, se decidió priorizar la incorporación de mecanismos que faciliten la interpretación de los eventos registrados y fortalezcan las capacidades de auditoría del sistema.

Decisión Técnica

Después de analizar ambas alternativas, se decidió utilizar la librería **local-time** en el desarrollo del proyecto.

La elección se basó en que esta herramienta aporta una mejora directa a una de las funcionalidades más importantes del sistema: el registro y seguimiento de los cambios de estado del semáforo. Gracias a su integración, la información generada resulta mucho más clara y sencilla de interpretar, lo que facilita el análisis de los eventos ocurridos durante la ejecución.

Además, trabajar con local-time permitió incorporar una librería externa al proyecto y conocer el proceso completo de instalación, configuración y utilización de paquetes mediante Quicklisp. Esto representó una oportunidad para ampliar los conocimientos sobre el ecosistema de Common Lisp y comprender cómo extender las capacidades de una aplicación utilizando herramientas desarrolladas por terceros.

Otro aspecto valorado fue la posibilidad de mejorar la presentación de los datos sin necesidad de realizar cambios importantes en la estructura general del sistema. De esta manera, se logró agregar una funcionalidad útil manteniendo la simplicidad y organización del código desarrollado.

En conclusión, se consideró que local-time era la opción más adecuada para complementar el proyecto, ya que aporta una mejora práctica, fácil de implementar y alineada con los objetivos planteados para esta etapa del trabajo.

FASE 3: ESTUDIO COMPARATIVO – ERLANG

Introducción al Lenguaje

Erlang es un lenguaje de programación funcional desarrollado por Ericsson a mediados de la década de 1980 para la construcción de sistemas de telecomunicaciones altamente concurrentes y tolerantes a fallos.

Su diseño se basa en los principios de programación funcional, inmutabilidad de datos y concurrencia mediante procesos ligeros que se comunican mediante envío de mensajes.

Una de sus características más destacadas es la capacidad de mantener sistemas funcionando durante largos períodos de tiempo sin interrupciones, motivo por el cual es ampliamente utilizado en aplicaciones donde la disponibilidad es crítica.

Áreas de Aplicación

Erlang suele utilizarse en sistemas que requieren:

- Alta disponibilidad.
- Procesamiento concurrente.
- Escalabilidad.
- Tolerancia a fallos.
- Comunicaciones en tiempo real.

Algunos ejemplos son:

- Redes de telecomunicaciones.
- Sistemas bancarios.
- Plataformas de mensajería.
- Infraestructura de servidores.
- Sistemas distribuidos.

Empresas que Utilizan Erlang

Diversas organizaciones han utilizado Erlang en sistemas de producción debido a sus capacidades de concurrencia y confiabilidad.

Entre ellas pueden mencionarse:

- Ericsson
- WhatsApp
- Klarna
- RabbitMQ
- Discord

Implementación de la función transición en Erlang

Para representar el comportamiento del semáforo en Erlang se implementó la función `transicion/1`.

La solución aprovecha una de las características más importantes del lenguaje: la coincidencia de patrones (pattern matching). Cada estado posible del semáforo se encuentra representado mediante una cláusula independiente que devuelve el siguiente estado correspondiente dentro del ciclo.

Los estados considerados son:

- en-rojo
- intermitente-rojo
- en-verde
- intermitente-verde
- en-amarillo

- intermitente-amarillo

Además, se incorporó un caso por defecto que permite detectar estados inválidos.

Esta estrategia permite construir una solución clara, sencilla y alineada con el paradigma funcional promovido por Erlang.

Implementación de la función timer en Erlang

Para replicar el funcionamiento del temporizador desarrollado en Common Lisp se implementó una función capaz de obtener el tiempo actual del sistema y determinar el estado correspondiente del semáforo.

La implementación utiliza la función `os:system_time(seconds)` para obtener el tiempo Epoch actual y posteriormente calcula la posición del instante dentro del ciclo semafórico mediante el operador resto (`rem`).

El ciclo completo posee una duración total de 225 segundos, considerando tanto los colores principales como los estados de intermitencia incorporados en la Iteración 2.

Posteriormente, la función `obtener_color/1` utiliza guardas (`when`) para determinar qué estado corresponde al instante calculado.

De esta forma, la solución mantiene la misma lógica implementada en Common Lisp, pero utilizando mecanismos propios de Erlang como el pattern matching y las guardas para la toma de decisiones.

Pregunta 1

Delimitación de Funciones y Expresiones en Erlang

Una diferencia importante entre Common Lisp y Erlang es la forma en que ambos lenguajes representan la estructura del programa.

En Common Lisp la organización del código se realiza mediante listas delimitadas por paréntesis:

```
(defun timer (timestamp)
```

```
...)
```

En Erlang se utilizan distintos símbolos para indicar el fin de expresiones y cláusulas.

Coma (,)

Se utiliza para separar expresiones que pertenecen a una misma cláusula.

Posicion = Timestamp rem Ciclo,

Color = rojo

Punto y coma (;)

Se utiliza para separar cláusulas alternativas dentro de una misma función.

transicion(en_rojo, verde) ->

...

;

transicion(en_verde, amarillo) ->

...

Punto (.)

Indica el final definitivo de una función.

transicion(Estado, _) ->

{Estado, accion_por_defecto}.

Al provenir de Common Lisp, la lectura inicial del código resultó menos intuitiva debido a la ausencia de paréntesis como estructura principal. Sin embargo, el uso de pattern matching hace que la lógica de las funciones resulte muy clara una vez comprendida la sintaxis.

Pregunta 2

Variables de Asignación Única y Simulación de un Ciclo Continuo

Una característica fundamental de Erlang es que las variables poseen asignación única.

Esto significa que una vez que una variable recibe un valor, dicho valor no puede modificarse posteriormente.

Por ejemplo:

$X = 10$.

Luego no es posible hacer:

$X = 20$.

Debido a esta restricción, los programas no utilizan variables mutables para representar cambios de estado.

En lugar de modificar variables, Erlang modela los cambios mediante recursión y paso de parámetros.

Un ciclo continuo del semáforo podría implementarse de forma conceptual de la siguiente manera:

ciclo(Estado) ->

NuevoEstado = siguiente_estado(Estado),

ciclo(NuevoEstado).

En cada llamada recursiva se crea un nuevo estado que es enviado como argumento a la siguiente ejecución.

De esta forma el sistema evoluciona sin necesidad de modificar ninguna variable existente, manteniendo los principios de inmutabilidad característicos de la programación funcional.

Comparación con Common Lisp

Tanto Erlang como Common Lisp permiten desarrollar soluciones funcionales para el problema del semáforo.

Sin embargo, Erlang enfatiza fuertemente la concurrencia y la comunicación entre procesos, mientras que Common Lisp ofrece una sintaxis más flexible y generalista.

La principal diferencia observada durante la implementación fue el uso intensivo de pattern matching en Erlang, lo que permite expresar distintas transiciones del semáforo mediante múltiples cláusulas de función en lugar de estructuras condicionales más extensas.

Conclusión

La experiencia de utilizar Erlang permitió conocer un lenguaje funcional orientado a sistemas concurrentes y distribuidos.

Aunque su sintaxis resulta diferente a la de Common Lisp, comparte principios fundamentales como la inmutabilidad y la ausencia de estados mutables.

El uso de pattern matching simplifica la representación de las transiciones del semáforo y demuestra una forma declarativa de resolver problemas de control de estados.

Además, el estudio de Erlang permitió comprender cómo sistemas reales de alta disponibilidad pueden modelar procesos continuos sin recurrir a variables mutables, utilizando recursión y comunicación entre procesos como mecanismos principales de ejecución.

CONCLUSIÓN GENERAL

El desarrollo del Sistema de Semáforos Inteligentes permitió aplicar de manera práctica los conceptos fundamentales de la programación funcional mediante la utilización de Common Lisp para resolver una problemática inspirada en situaciones reales de control de tránsito.

A lo largo del proyecto se implementaron mecanismos de transición de estados, temporización automática, auditoría de eventos, análisis de ciclos y generación de estadísticas de funcionamiento. Posteriormente, el sistema fue ampliado mediante la incorporación de estados de intermitencia y la generación de informes persistentes en archivos de texto.

La integración de la librería local-time permitió mejorar significativamente la presentación de la información registrada, facilitando la interpretación de los eventos ocurridos durante la ejecución del sistema y fortaleciendo las capacidades de auditoría requeridas por la consigna.

Asimismo, el estudio comparativo realizado con Erlang permitió conocer otra alternativa dentro del paradigma funcional y analizar diferentes formas de modelar una misma problemática utilizando herramientas y mecanismos propios de cada lenguaje.

En conjunto, el trabajo permitió consolidar conocimientos sobre programación funcional, diseño de algoritmos, depuración de software e integración de bibliotecas externas, demostrando la aplicabilidad de estos conceptos en el desarrollo de soluciones para problemas del mundo real.

BIBLIOGRAFÍA

- Common Lisp HyperSpec:
<https://www.lispworks.com/documentation/HyperSpec/>
- Quicklisp: <https://www.quicklisp.org>
- CL-JSON: <https://cl-json.common-lisp.dev>
- local-time: <https://github.com/dlowe-net/local-time>
- Common Lisp Documentation: <https://common-lisp.net>
- Erlang Documentation: <https://www.erlang.org/docs>
- Erlang Programming Language: <https://www.erlang.org>
- Material de la Cátedra Paradigmas y Lenguajes de Programación 2026.